

Key Point Summary: Indexing Structures & Physical Database Design

Focus: HOW each structure actually *functions* (search / insert / lookup mechanics)

1. The Core Idea Behind Every Index

Function: An index is a *separate, smaller file* that sits beside the data file and provides a shortcut path to records, without touching the physical placement of records in the data file itself.

How a lookup works, in general:

1. Search the (small, ordered) index file — usually via binary search.
2. The index entry found gives you a **pointer** (block address, or block+offset).
3. Follow that pointer into the (large) data file to fetch the actual record.

This is exactly like a book's back-of-book index: you search the index (alphabetical, small) instead of reading the whole book (linear, large), then jump straight to the page.

Why it works: the index file is *much smaller* than the data file (fewer entries, and each entry is small — just a key + a pointer), so a binary search over it costs far fewer block accesses than searching the data file directly.

2. Primary Index — How It Functions

Precondition: the data file is physically sorted on disk by a **key** field (unique per record).

Functioning mechanism:

- The index stores **one entry per data block**, not per record.
- Each entry = $\langle K(i), P(i) \rangle$ = the key of the **block anchor** (the first record in block i) + a pointer to block i .
- Because the file is physically sorted, *every record in block i has a key $\geq K(i)$ and $< K(i+1)$.*

Search procedure (step-by-step):

1. Binary search the index for the entry where $K(i) \leq \text{search_key} < K(i+1)$.
2. Follow $P(i)$ to load block i from the data file.
3. The desired record is guaranteed to be inside that block — scan it directly.

Why insertion is awkward: inserting a new record means shifting other records to keep the file sorted, which can shift block anchors, which means index entries must also be updated. Fix: use an **unordered overflow file** or a **linked overflow list** per block, so the main file/index doesn't need reshuffling on every insert.

3. Clustering Index — How It Functions

Precondition: the data file is physically sorted by a **nonkey** field (many records share the same value — e.g., Department_number).

Functioning mechanism:

- One entry **per distinct value** of the clustering field (not per block, not per record).
- Each entry points to the **first block** containing that value.
- All records sharing that value are physically grouped together right after that point.

Search procedure:

1. Binary search the index for the target value.
2. Follow the pointer to the first block with that value.
3. **Scan forward** through that block (and subsequent blocks if the group spans more than one block) to collect all matching records.

Insertion problem & fix: inserting into the middle of a sorted nonkey group is still disruptive. The standard fix: **reserve a whole block (or cluster of blocks) per distinct value** in advance, so new records for that value just drop into the reserved space — no reshuffling needed elsewhere.

4. Secondary Index — How It Functions

Precondition: built on a field that does **NOT** determine the file's physical order (could be key or nonkey).

Functioning mechanism (key field case — dense):

- Because the data file is *not* sorted by this field, there's no guarantee about *where* a record with a given value sits relative to its neighbours.
- So the index **must** have one entry per *record* (not per block) — otherwise some records would be unreachable.
- Each entry = $\langle K(i), P(i) \rangle$, ordered by $K(i)$, where $P(i)$ is a block (or record) pointer.

Search procedure:

1. Binary search the (large) index for the exact key value.
2. Follow the pointer straight to the record (or its block).

Functioning mechanism (nonkey field case — duplicates exist):

Three implementation options, each with a different functioning style:

- **Option 1:** one index entry *per record*, duplicates allowed → simple, but dense and large.
- **Option 2:** one index entry per *distinct value*, but the entry holds a **variable-length list of pointers** (one per matching record).
- **Option 3 (most common):** keep index entries fixed-length (one per distinct value), but each entry's pointer leads to an **extra block of record-pointers** (a level of indirection) — that block lists all the actual record locations for that value.
- *Why this matters functionally:* fixed-length entries keep binary search simple and fast; the indirection block absorbs the variability of "how many records have this value."

Key trade-off vs. primary index: a secondary index needs **more storage and more search time** (it's dense, with many more entries) — but it's the *only* way to avoid a slow linear scan, since (unlike a primary-indexed file) there's no physical ordering to fall back on.

5. Multilevel Index — How It Functions

The problem it solves: binary search on a single-level index still needs $\log_2(b)$ block accesses. If the index itself has many blocks, that's still a lot of disk I/O.

Functioning mechanism — "treat the index as its own data file":

1. Take the first-level index (already ordered) and treat it *as if it were a data file*.
2. Build a **primary index on top of it** → this becomes the *second level*.
3. Repeat — build a primary index on the second level → third level, etc.
4. Stop once a level fits in a **single block** — this is the **top level**.

Why this is fast — the fan-out mechanism:

- Binary search reduces the search space by **2x** at each step (hence $\log_2$).
- Multilevel indexing reduces the search space by **fo (the fan-out)** at each step — and fan-out is usually in the hundreds (e.g., 273 in the textbook's worked example).
- So instead of $\log_2(b)$ accesses, you get $\log_{fo}(b)$ accesses — drastically fewer, because each level "jumps" much further than a binary split would.

Search procedure (general, t levels):

1. Start at the single top-level block — read it.
2. Find the entry where $K(i) \leq \text{search_key} < K(i+1)$; follow that pointer down one level.
3. Repeat at each level until you reach level 1 (the base index).
4. Follow the level-1 pointer into the actual data file block.
5. **Total block accesses = t (one per index level) + 1 (data block).**

Why it still has problems: every level is a **physically ordered file** — so insertions/deletions still cause the same reshuffling headaches as a single-level primary index, just now at multiple levels. This motivates the move to **dynamic** structures (B-trees / B+ -trees).

ISAM connection: IBM's ISAM is exactly this idea applied to disk geometry — a **cylinder index** (level 2) points to a **track index** (level 1) which points to a **track** (scanned sequentially) — i.e., multilevel indexing mapped onto physical disk layout.

6. B-Tree — How It Functions

The core innovation: unlike a static multilevel index, a B-tree **expands and shrinks dynamically** as data is inserted/deleted, while staying balanced and never collapsing the search benefit.

Node anatomy (order p):

$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \dots, P_q \rangle$ — each node mixes **tree pointers** (to children) *and* **data pointers** (to records) at every level, even internal ones.

Functioning rule — the "fullness contract":

- Every node (except root/leaf) must stay between **half full** ($\lceil p/2 \rceil$ pointers) and **completely full** (p pointers).
- This guarantees no node is ever nearly empty or overloaded — keeping the tree shallow and balanced.

Insertion mechanics (the part that actually "functions" dynamically):

1. Search down to the correct **leaf node** for the new key.
2. If the leaf has room → insert directly, done.
3. If the leaf is **full** → **split** it into two nodes at the same level; the middle value moves *up* to the parent (along with two new child pointers).
4. If the parent is now full too → it **also splits**, and the split **propagates upward**.
5. If the split reaches the root and the root is full → the root splits, and a **brand new root is created** — the tree gains a level.

Deletion mechanics (mirror image):

1. Remove the key from its leaf.
2. If this leaves the node **less than half full** → **merge** it with a sibling node.
3. Merging can propagate upward just like splitting — and can **reduce** the number of levels if the root ends up with only one child.

Steady-state behaviour (important exam fact): after many random insertions/deletions, nodes settle at roughly **69% occupancy** — full enough to be efficient, with just enough slack to absorb new insertions without triggering a split every time.

7. B+-Tree — How It Functions (and how it differs from a B-tree)

The one structural change that matters: data pointers are **removed from internal nodes** and pushed down so they exist **only in leaf nodes**.

- **Internal nodes** = pure "routing" nodes: just keys + tree pointers, no data pointers. Their only job is to guide the search downward.
- **Leaf nodes** = the only place where $\langle \text{key}, \text{data-pointer} \rangle$ pairs actually live, **plus** a "next" pointer linking leaf nodes left-to-right into a chain.

Why removing data pointers from internal nodes matters functionally:

Smaller internal-node entries → **more entries fit per node** → **higher fan-out** → **fewer levels** for the same number of records → **fewer block accesses per search**, compared to a B-tree of the same order.

Search procedure (Algorithm 17.2 logic):

1. Start at the root.
2. At each internal node, compare the search key K to the node's key values:
 - if $K \leq K_1$ → go to leftmost pointer P_1
 - if $K > K_{(q-1)}$ → go to rightmost pointer P_q
 - else find i such that $K_{(i-1)} < K \leq K_{(i)}$ → go to pointer P_i

3. Repeat until you reach a **leaf node**.
4. Scan the leaf for the matching key, then follow its data pointer to the record.

Insertion procedure (Algorithm 17.3 logic) — the "split and push up a copy" mechanic:

1. Search down to the correct leaf (just like a search), **pushing each parent node's address onto a stack** as you go (you'll need to retrace your steps if a split happens).
2. If the leaf has room → insert the **<K, Pr>** entry in sorted position. Done.
3. If the leaf is **full**:
 - Create a temporary oversized node holding all old entries **plus** the new one.
 - Split it into two leaf nodes: the **first half stays**, the **second half goes into a brand-new leaf node**.
 - Re-link the "next" pointers so the leaf chain stays intact.
 - **Copy** (not move) the smallest key of the new right-hand leaf **up into the parent** internal node, along with a pointer to the new leaf.
4. If the parent is now full → the parent **also splits** the same way, and this propagates up the stack toward the root.
5. If the root itself splits → a new root is created, adding a level to the tree.

Crucial B-tree vs. B+-tree insertion difference: in a B-tree, the *middle key itself moves up* (it disappears from the lower level). In a B+-tree, the key that "moves up" to the parent is **copied**, not removed — it still exists down in the leaf level too, since that's the only place where the actual data pointer lives.

Why B+-trees dominate in practice: higher fan-out (fewer levels, fewer block accesses) **and** the linked leaf chain makes **range queries and sequential scans** (**BETWEEN, ORDER BY**) very efficient — you just walk the leaf chain instead of re-searching the tree for each value in the range.

8. Multiple-Key Access Structures — How Each One Functions

(a) Composite (Multi-Attribute) Ordered Index

Mechanism: treat the combination of attributes **<A1, A2, ..., An>** as one single search key, and order entries **lexicographically** (like sorting strings) — e.g., **<3, 99>** comes before **<4, 1>**. Functions exactly like a normal single-level/multilevel index once you treat the tuple as "one value."

(b) Partitioned Hashing

Mechanism:

1. Each component of the composite key is hashed **separately** into its own fixed-length bit address (e.g., Dno → 3 bits, Age → 5 bits).
2. The two hash outputs are **concatenated** into one combined bucket address (3+5 = 8 bits total).
3. To search on **both** keys: compute both hash parts, concatenate, go straight to that one bucket.
4. To search on **only one** key (e.g., Age only): you must scan **every bucket** that has the matching bits in the Age portion, regardless of the Dno portion — i.e., search all $2^3 = 8$ possible Dno prefixes.

Functional limitation: hashing scrambles order, so there is **no way to do range queries** — you can't ask for "all Ages between 30–40" because hashed values for 30–40 aren't stored contiguously.

(c) Grid Files

Mechanism:

1. Build one **linear scale** per search attribute (e.g., one scale for Dno, one for Age), where each scale groups raw values into buckets designed for *roughly even distribution* (not equal-width — equal population).
2. The combination of (Dno-scale-position, Age-scale-position) defines a **cell** in an n-dimensional **grid array**.
3. Each cell points to a **bucket** holding the matching records.

Search procedure: convert your query values into their scale positions → look up that exact cell → follow its pointer to the bucket.

Why it's good for range queries (unlike partitioned hashing): a range query (e.g., "Dno ≤ 5 AND Age > 40") maps to a **contiguous block of cells** in the grid — so you can identify and fetch only the relevant set of buckets directly, since the grid preserves order along each scale.

Trade-off: the grid array itself is extra storage overhead, and it needs **frequent reorganization** as data changes — costly for highly dynamic files.

(d) Hash Index

Mechanism: essentially a hash-table-based version of a secondary index. Entries $\langle K, Pr \rangle$ are stored in a dynamically expandable hash file; you hash the search key K to compute the bucket, then look inside that bucket for the matching entry and follow its pointer.

Function vs. limitation: very fast for **equality** lookups; like all hash structures, it has **no ordering**, so it cannot support range queries.

(e) Bitmap Index

Mechanism (the part people most often get wrong):

1. For a column C with m distinct values, build **m separate bit-vectors**, each of length n (n = number of rows in the relation).
2. Vector for value V: bit i = 1 if row i has value V for column C, else 0.

How queries function using bitmaps:

- **Equality** ($C = V$): just read the one bitmap for V — the 1-bits are your matching Row_ids.
- **AND** ($C1=V1$ AND $C2=V2$): fetch both bitmaps, do a bitwise **AND** (intersect) → result bitmap's 1-bits are the rows matching *both* conditions.
- **OR** ($C1=V1$ OR $C2=V2$ OR $C3=V3$): fetch all three bitmaps, do a bitwise **OR** (union).
- **NOT** ($C1 \neq V1$): take the complement (flip every bit) of the bitmap for V1.
- **COUNT**: just count the 1-bits in the relevant bitmap — no need to touch the data file at all.

Why this is fast: bitwise AND/OR/NOT on 32- or 64-bit chunks are **single CPU instructions** — so combining multiple conditions is extremely cheap computationally, and the storage is tiny (1 bit per row per value) when the column has **few distinct values** relative to row count.

When it stops being efficient: if a column has very few distinct values overall (e.g., Sex with only M/F), each bitmap is "dense" (about 50% of rows are 1) — at that point, just scanning the whole table directly is competitive or better than bitmap filtering.

Bitmaps inside B+-tree leaves: when a nonkey value occurs in a *very large fraction* of records, a B+-tree leaf can store a **bitmap instead of a long list of record pointers** for that value — because the bitmap ($n/8$ bytes) ends up smaller than storing thousands of individual 4-byte pointers once the value's frequency crosses a certain threshold (worked example in the textbook: crossover around 1/32 frequency).

(f) Function-Based Index

Mechanism: instead of indexing the raw column value, the index is built on the **result of applying a function** to the column (e.g., `UPPER(Lname)`).

How it functions at query time: when a query's WHERE clause applies the *same function* to the column (e.g., `WHERE UPPER(Lname) = 'SMITH'`), the DBMS recognizes this matches the function-based index definition and uses the index directly — **avoiding a full table scan** that would otherwise be forced by the presence of a function wrapped around the column.

(g) Logical vs. Physical Index

Physical index: entry $\langle K, P \rangle$ — P is an actual disk address (block + offset). **Functional weakness:** if the record physically moves (e.g., a hash bucket splits in extendible hashing), every index pointing to that record must be **found and updated** — expensive and error-prone.

Logical index: entry $\langle K, K_p \rangle$ — instead of a physical address, it stores the **primary key value** of the record. **How a lookup functions:** search the logical index → get K_p → now do a **second lookup** using K_p through the file's *primary* access method (e.g., its primary index or hash function) to find the actual current physical location. **Trade-off:** one extra lookup step (the cost of indirection) in exchange for **never having to fix pointers** when records physically relocate.

9. Physical Database Design — How the Decision Process Functions

The design process, step by step:

1. **Profile the job mix** — for every expected query: which files, which selection attributes (equality/range), which join attributes, which retrieved attributes. For every update: which files, operation type, selection attributes (for WHERE), and which attributes get *modified*.
2. **Weight by frequency** — apply the 80/20 rule: focus index design on the ~20% of queries/transactions responsible for ~80% of load: don't index for rare, low-impact queries.
3. **Flag time-critical paths** — queries/transactions with strict SLAs get bumped to the front of the queue for the *best* access structure available (usually the primary/clustering index, since it's the cheapest to use).
4. **Check update cost** — every index added is an index that must be *maintained* on every insert/update/delete. A heavily-written table should carry the **minimum** indexes that are actually justified.
5. **Force-index uniqueness constraints** — every candidate key / UNIQUE attribute gets an index **regardless of query patterns**, because that's the only efficient way for the DBMS to *check* the constraint without a full scan on every insert.

Then the actual decisions get made, functioning as follows:

- **Whether to index at all:** only if the attribute is a key, OR is used in a selection/join condition somewhere in the profiled job mix.
- **Single vs. composite index:** if multiple attributes are *consistently queried together*, build one composite index — but the **attribute order inside the index must match the query pattern** (e.g., index on `<Garment_style, Color>` only helps if queries filter by style first, then color — not the reverse).
- **Which index (if any) gets to be the clustering/primary index:** only ONE per table is possible (physical ordering constraint, same as in Section 2/3 above) — pick the attribute that benefits most from **range queries**, and avoid clustering an attribute that's meant to be answered by **index-only scans** (clustering's main benefit is *fetching the actual records faster*, which doesn't matter if you never touch the records).
- **Hash vs. tree index:** choose hash if the workload is **equality-only** (e.g., join lookups); choose B+-tree (the default in most systems) if **range queries** are needed too, since hashing structurally cannot support them.

One-Paragraph "Explain It Out Loud" Summary (for teaching)

"Every index works the same basic way: it's a small, separate, sorted file of `<key, pointer>` pairs that you binary-search instead of scanning the whole data file. A **primary index** only needs one entry per *block* because the file is sorted by that key. A **clustering index** is the same idea but for a repeated (nonkey) value. A **secondary index** can't rely on physical order at all, so it needs an entry for *every record* (if dense) — or a layer of indirection if many records share a value. When even the index gets too big, you stack another index on top of it — that's a **multilevel index** — and you keep stacking until one block holds everything: that's the top. The problem is, ordered files hate insertions. So real systems use **B+-trees**: a tree that splits nodes when they overflow and merges them when they're too empty, always staying balanced, with all the actual record-pointers pushed down into linked leaf nodes so that internal levels can pack in more keys and keep the tree shallow. For queries on **multiple keys at once**, you either treat the combo as one sorted key, hash each part separately (fast but no ranges), or use a grid (*supports ranges but costly to maintain*). And for columns with **few distinct values**, you skip pointers entirely and just store a bitmap — because ANDing and ORing bits is something the CPU does almost for free."

Pair this with the earlier exam practice document: use that one to test recall of facts/definitions, and this one to test whether you can explain the **mechanics** — which is usually where the harder exam marks (and the best teaching moments) live.